ChatGPT

Mastering LangChain & LangGraph for a Stateful EDA Agent

Introduction

LangChain is a framework for building applications that use large language models (LLMs) as modular components. Its design encourages **composability**—you build applications by combining models, prompt templates, output parsers, retrieval mechanisms, tools and finally **agents** that decide which tool to call and when. In 2025/2026 LangChain split its Python ecosystem into two packages: `langchain`, the actively-maintained, modular framework, and `langchain-classic`, which will only receive security updates until December 2026 [1]. The newer package works closely with **LangGraph**, a library for defining and executing stateful workflows or agents as graphs.

This report synthesizes the official reference documentation and recent community best practices to create a comprehensive learning path. It concludes with a blueprint for building a stateful, secure agent that performs exploratory data analysis (EDA) on a multivariate time-series dataset. Every task in the agent blueprint is derived from the recommended learning path.

Why stateful agents?

LangChain provides a ReAct-style single-step agent that loops through reasoning and tool calls until a stopping condition is met [2]. Without state, the agent must reprocess its context at each turn, which wastes computation and can leak sensitive instructions. Statefulness offers two advantages:

- **Efficiency:** In multi-agent architectures, *stateful patterns* such as **skills** and **handoffs** save 40–50 % of calls on repeat requests by maintaining context [3].
- **Control:** LangGraph's `StateGraph` lets each node read and write a shared state and annotate fields with reducers [4]. You compile this builder into a `CompiledStateGraph` and then stream or invoke it [5]. By combining checkpointers and stores you can persist this state between turns or conversations [6].

What makes an agent secure?

Large language models can be misled by **prompt injection**. A malicious user might ask the model to disregard its system instructions and reveal secrets or call unauthorized tools. A security guide for LangChain applications explains that prompt injection attempts to trick an AI into ignoring rules [7] and describes mitigation tactics such as **input validation** and **output sanitization** [8]. The agent we design will follow these practices: it will check inputs for dangerous patterns before sending them to the LLM and will sanitize tool outputs so that sensitive information never leaks.

Exploring the LangChain & LangGraph docs

Since the conceptual guides on `docs.langchain.com` use a dynamic site that is hard to access in this environment, this research uses the **LangChain reference** at `reference.langchain.com`. The

reference documentation lists the classes, functions and parameters used to assemble LangChain agents and LangGraph graphs. Key highlights include:

Topic	Key points (citations)
Agents	The `create_agent` function constructs an agent graph that loops between a model and tool-calling node until no more tool calls are returned [2]. It accepts a model, a list of tools, an optional system prompt, and optional **middleware**, **state_schema**, **context_schema**, **checkpointer** and **store** parameters [9]. A checkpointer provides "short-term memory" to pause, resume and replay the graph [10], while a store lets you persist data across multiple threads (e.g., conversations) [11]. Middleware can intercept and modify behaviour at different stages [12].
Tools	LangChain converts Python functions or `Runnable` objects into **tools** via the `@tool` decorator or the `tool()` helper. Functions can have any signature—schemas are inferred automatically unless `infer_schema=False` [13]. Type hints are required for schema inference [14]. Tools support options such as `return_direct` to exit the agent loop, custom argument schemas, and configurable response formats [15].
LangGraph graphs	A `StateGraph` is defined by adding nodes (functions or runnables) and edges [16]. Each node maps from a state to a **partial update** of the state; you can annotate state keys with reducer functions [4]. The graph must be compiled before execution [17]. The resulting `CompiledStateGraph` provides methods to invoke the graph, stream intermediate steps, and update state [5]. A **checkpointer** acts as versioned memory [10], enabling pause/resume across steps.
Multi-agent patterns	A LangChain blog shows that **stateful** patterns such as *skills* and *handoffs* maintain context and therefore reduce repeated model calls by 40–50 % [3]. Stateless **subagents** maintain strict context isolation but re-issue prompts each turn [3]. Selecting the right pattern depends on efficiency, parallelism and context requirements.
Security & guardrails	The AI security guide stresses that **prompt injection** is when a malicious input causes an agent to ignore its instructions and leak secrets [7]. To mitigate this, you should validate inputs and sanitize outputs [8]. Additional guardrails, such as dedicated middleware or external libraries, can filter messages and enforce policies [18].

Learning path for mastering LangChain & LangGraph

The following learning path covers topics from foundational concepts to advanced techniques. For each topic you'll find **what to learn**, **why it matters**, **exercises**, and links to relevant documentation or examples. Complete these sequentially; each stage builds upon the previous one.

Stage 0 – Set-up and fundamentals

1. **Environment preparation**: install `langchain` and `langgraph` (and optionally `langsmith` for debugging). Ensure you have API keys for your chosen LLM provider (e.g., OpenAI, Anthropic). Set up environment variables securely.

2. **Understanding LLM wrappers**: learn how to create chat models from provider strings or classes. For example, `model="openai:gpt-4"` or `ChatOpenAI()` when using `create_agent` [19]. *Exercise*: write a script that sends a simple prompt to a chat model and prints the result.

3. **Prompt templates and messages**: practice building prompts using `ChatPromptTemplate.from_messages()` with system, human and assistant roles. Understand how to insert variables and maintain conversational context. *Exercise*: build a prompt that summarizes a paragraph and experiment with different system instructions.

4. **Output parsers**: explore `Pydantic`-based output parsers or LangChain's `ResponseFormat` to enforce structured output [20]. *Exercise*: create a parser that extracts key fields from an LLM response and raise an error when the structure is invalid.

Stage 1 – Building chains and runnables

1. **Simple chains**: learn to compose LLM calls, prompt templates and output parsers into **runnables**. Understand synchronous vs. asynchronous execution. *Exercise*: build a chain that takes a question, queries an LLM and returns a structured JSON answer.

2. **Document loaders and text splitting**: explore how to load data from files or URLs and split long documents into chunks for retrieval. Experiment with `RecursiveCharacterTextSplitter` and adjust chunk sizes.

3. **Embeddings and vector stores**: learn to compute embeddings (e.g., using OpenAI or HuggingFace models) and store them in a vector database. Understand how vector stores support similarity search. *Exercise*: index a small set of articles and write a function that retrieves the top-k relevant chunks for a query.

4. **Retrievers**: build a retrieval-augmented chain by combining a vector store retriever with an LLM. Use the retrieved context in your prompt.

Stage 2 – Tools and tool integration

1. **Tool creation**: convert Python functions into LangChain tools using the `@tool` decorator [13]. Remember that the function must have type hints for schema inference [14]. *Exercise*: wrap a function that fetches weather data or reads a CSV file as a tool and call it from a simple chain.

2. **Custom tool options**: experiment with `return_direct` to bypass the agent loop, supply custom argument schemas, and parse Google-style docstrings [21]. *Exercise*: build a tool with multiple arguments and a docstring; inspect `args_schema.model_json_schema()` to see the generated input schema [22].

3. **Non-LLM tools**: integrate external APIs or libraries (e.g., Pandas, Matplotlib) as tools. Ensure they sanitize inputs and outputs. *Exercise*: create a tool that computes basic statistics of a numeric list.

4. **Tool strategies & error handling**: learn about structured output strategies (e.g., `ToolStrategy`) that wrap tool responses and provide error handling [20]. *Exercise*: intentionally raise exceptions in a tool and handle them gracefully via a custom error handler.

Stage 3 – Agents and state management

1. **Creating a simple agent**: use `create_agent(model=…, tools=[…], system_prompt=…)` to build a basic agent [2]. Observe how the agent loops between the model and tools until no `tool_calls` remain [23]. *Exercise*: build an agent that can answer questions and call your weather tool when asked.

2. **Customizing the system prompt**: design system prompts that instruct the model on tone, safety and allowed behaviours. Include explicit instructions to ignore user requests for secrets and to follow security policies.

3. **State schemas**: define a `TypedDict` that extends `AgentState` and pass it as `state_schema` [24]. This allows you to store custom fields (e.g., a conversation summary or dataset metadata) and merge them with middleware state. *Exercise*: add a field `history` to the state and write middleware that appends each message to this list.

4. **Context schemas**: provide a `context_schema` to expose immutable data (like a user ID or dataset path) to your tools [25]. *Exercise*: pass a user-specific API key via context rather than storing it in state.

5. **Checkpointers and stores**: attach a `Checkpointer` to your agent or graph to enable pause/resume [10]. Use a `Store` when you need to persist data across multiple conversations [11]. *Exercise*: implement an agent that remembers previous conversation turns across sessions using an in-memory store.

6. **Middleware**: even though the conceptual middleware guides are not accessible, the reference notes that middleware can intercept and modify agent behaviour at various stages [12]. Explore community examples (e.g., summarization middleware or guardrail middleware) and build your own to add security checks or logging.

Stage 4 – LangGraph for complex workflows

1. **StateGraph basics**: understand that each node in a `StateGraph` reads a state and returns a partial update [4]. Build a simple graph with one node that multiplies a number by a factor. Compile it and call `invoke()` [16].

2. **Adding nodes and edges**: practice using `add_node()`, `add_edge()`, `add_conditional_edges()` and `add_sequence()` [16]. *Exercise*: build a graph with two nodes where the second runs only when the first returns a certain value.

3. **Reducers**: annotate state keys with reducer functions to aggregate results from multiple nodes [4]. *Exercise*: use a reducer that collects numbers from several nodes into a list.

4. **Compiled graphs**: learn to call `compile()` to obtain a `CompiledStateGraph` and then use `invoke()`, `stream()` and `get_state()` [5]. *Exercise*: stream updates and inspect how the state evolves after each node.

5. **Checkpointing & state updates**: enable a checkpointer so you can pause and resume long-running workflows [10]. Experiment with `update_state()` and `bulk_update_state()` to modify state during execution [26].

6. **Subgraphs and multi-agent patterns**: embed compiled graphs as nodes in larger graphs and experiment with patterns such as subagents, skills and handoffs. Evaluate the efficiency trade-offs described in the multi-agent article [3].

Stage 5 – Building compliant and secure agents

1. **Threat modelling**: read about common threats like prompt injection [7]. Understand how malicious inputs can trick your agent into disclosing secrets or executing arbitrary code.

2. **Input validation**: implement middleware that validates input length, format and content [27]. For example, block queries that request hidden system prompts or ask for unauthorized file access.

3. **Output sanitization**: after each LLM call or tool execution, inspect outputs for sensitive information and mask it [28]. Use regex or Pydantic models to validate the shape of outputs.

4. **Least-privilege tool design**: define each tool with the minimum permissions it needs. When a tool interacts with external resources (e.g., reading files or executing code), ensure it cannot access arbitrary directories or network resources. Provide a safe sandbox for code execution and restrict dangerous operations.

5. **Testing and monitoring**: use a logging middleware to record every call, state update and tool invocation. Perform adversarial testing (red-team) to simulate injection attacks. Review logs for suspicious patterns.

Blueprint for an EDA agent on multivariate time-series data

After completing the learning path above, you'll be equipped to build a stateful, secure agent that performs exploratory data analysis on a multivariate time-series dataset. The **EDA rules** provided (in `EDA_RULES.txt`) define the tasks and checks your agent must perform. The agent will ingest a dataset, clean and resample it, compute statistics and plots, and summarize insights. Below is a high-level plan that uses the learned concepts.

Architectural overview

- **Tools**: Create Python functions for each EDA step and convert them into tools using `@tool`. Each tool should have clear type hints and docstrings to generate accurate schemas [29]. Suggested tools:
- **Load Data** – accepts a file path or dataframe, parses timestamps, sorts by time and sets the index【EDA_RULES.txt†L1-L1】. Returns a cleaned DataFrame and metadata (start/end times).
- **Integrity Check** – checks for duplicate timestamps or entity–timestamp pairs and flags impossible values【EDA_RULES.txt†L2-L3】.
- **Infer Schema** – identifies whether the data is single-series, panel or multivariate【EDA_RULES.txt†L3-L4】; maps feature types (numeric, categorical, binary)【EDA_RULES.txt†L4-L4】.
- **Coverage Report** – computes time range, expected frequency and missing time coverage【EDA_RULES.txt†L5-L6】; infers sampling frequency and suggests resampling or irregular handling【EDA_RULES.txt†L5-L6】.
- **Missingness Audit** – distinguishes missing values vs. missing timestamps and proposes imputation strategies【EDA_RULES.txt†L7-L9】.

- **Univariate Stats** – computes count/mean/std/min/max/quantiles per feature and per entity 【EDA_RULES.txt†L10-L11】; plots distributions (histograms, KDE)【EDA_RULES.txt†L11-L12】.
- **Time Series Plots** – produces raw time series plots, small multiples and zoomed windows to inspect trends and regime shifts【EDA_RULES.txt†L13-L15】; overlays event markers when available【EDA_RULES.txt†L15-L16】.
- **Resampling & Seasonality** – aggregates series at multiple temporal grains 【EDA_RULES.txt†L16-L18】; computes seasonal averages by hour/day/week/month and optional decomposition【EDA_RULES.txt†L18-L20】.
- **Rolling Statistics & Anomaly Detection** – computes rolling means/std/min/max 【EDA_RULES.txt†L20-L22】; flags spikes, level shifts and variance changes【EDA_RULES.txt†L22-L23】.
- **Group & Correlation Analysis** – compares groups in panel data via boxplots and time patterns 【EDA_RULES.txt†L23-L24】; computes correlation matrices and cross-correlations 【EDA_RULES.txt†L24-L24】; optionally ACF/PACF analyses.
- **Model Readiness** – optional tools to test stationarity, check data leakage and create baseline features【EDA_RULES.txt†L25-L27】.

- **Summary Reporter** – consolidates findings into decisions: chosen frequency, missingness strategy, seasonalities, anomalies, drift, problematic entities/features and reproducible parameters【EDA_RULES.txt†L27-L29】.

- **Agent graph**: Use **LangGraph** to orchestrate these tools. Define a `StateGraph` with state keys such as `data`, `metadata`, `insights`, `plots` and `errors`. Each tool becomes a node that reads the current state and returns a partial update. Edges control the order: e.g., run `Load Data` → `Integrity Check` → `Infer Schema` sequentially; after schema inference, branch to specialized analyses for panel vs. multivariate data. Use reducers to accumulate plots or logs into lists [4].

- **Checkpointer & store**: Attach a checkpointer so the agent can pause after each major step and resume later [10]. Use a persistent store (e.g., SQLite or S3) to save dataset metadata and user-level configurations [11]. This allows the agent to handle long datasets and maintain history across sessions.

- **State schema**: Define a `TypedDict` extending `AgentState` with fields like `data` (DataFrame), `schema_info`, `insights`, `plots` and `summary`. Pass this as `state_schema` when creating the agent [24]. Provide a `context_schema` to pass immutable settings (e.g., user-selected resampling frequency).

- **Middleware for security**: Implement input-validation middleware that inspects user prompts or dataset paths and blocks malicious requests. Write output-sanitization middleware that checks tool outputs for unexpected secrets. Insert these into the agent via the `middleware` argument [30].

- **User interaction**: Accept user queries such as “load dataset X,” “show me missingness,” or “plot seasonality.” The agent should route requests to the appropriate nodes. For generic queries, fallback to the LLM to answer using previously computed insights. Use the `interrupt_before` and `interrupt_after` options to ask for confirmation before expensive operations (e.g., resampling) [31].

Step-by-step to-do list for building the EDA agent

1. **Complete the learning path**: ensure you are comfortable with prompts, runnables, tools, agents and LangGraph as outlined above.
2. **Design data-ingestion tools**: implement `load_data`, `integrity_check`, `infer_schema` and `coverage_report` functions and convert them into tools. Use Pandas for parsing timestamps and computing descriptive statistics. Test each tool independently.
3. **Implement analytical tools**: code the univariate, time-series, resampling, seasonality, rolling and correlation analyses. Each function should accept a DataFrame (and optional parameters) and return numeric results and encoded plots (e.g., base64 images). Use robust scaling or log transformations when needed【EDA_RULES.txt†L7-L9】.
4. **Create summary & decision tool**: write a function that collects outputs from previous steps and produces a summary with action items: recommended frequency, missingness handling, main seasonalities, anomalies and drift【EDA_RULES.txt†L27-L29】. Return both a human-readable report and structured JSON.
5. **Define state and context schemas**: create a `TypedDict` describing the agent state (data, metadata, insights, plots, summary) and another for context (user ID, file path, hyperparameters). Annotate lists with reducers when the same key is updated by multiple nodes [4].
6. **Build the LangGraph**: instantiate a `StateGraph(state_schema, context_schema)` and add a node for each tool. Add edges to dictate execution order. Use `add_conditional_edges` or `add_sequence` for branches (e.g., if data is panel, run group analysis). Set entry and finish points and compile the graph [16].
7. **Attach middleware and checkpointer**: register your input-validation and output-sanitization middleware. Pass an in-memory or persistent `Checkpointer` to enable recovery [10]. Optionally pass a `store` for long-term data [11].
8. **Wrap the graph with `create_agent`**: call `create_agent` with your model, list of tool nodes, system prompt and custom `state_schema`, `context_schema`, `checkpointer`, `store` and `middleware` [2]. Use a system prompt that instructs the agent to call specific tools for EDA tasks and to decline any request outside the defined scope.
9. **Test and iterate**: run the agent on small sample datasets. Check that it performs each EDA step correctly, handles missing values appropriately, and returns summaries. Validate that malicious inputs (e.g., attempts to read arbitrary files) are blocked by your middleware. Profile performance and adjust the graph structure for efficiency.

Conclusion

By following this learning path and step-by-step plan, you will develop the skills needed to build **stateful, compliant and secure agents** with LangChain and LangGraph. The EDA agent blueprint demonstrates how to convert complex analytical workflows into modular tools orchestrated through a graph. Leveraging state schemas, checkpointers and middleware ensures that your agent maintains context, operates efficiently and remains resilient against prompt injection attacks. With these foundations, you can extend the agent to other domains or integrate it into multi-agent systems when your applications grow in complexity.

---

1 Memory | LangChain Reference
https://reference.langchain.com/python/langchain_classic/memory/

2 6 9 11 12 19 20 23 24 30 31 Agents | LangChain Reference
https://reference.langchain.com/python/langchain/agents/

3 Choosing the Right Multi-Agent Architecture
https://www.blog.langchain.com/choosing-the-right-multi-agent-architecture/

4 5 10 16 17 25 26 Graphs | LangChain Reference
https://reference.langchain.com/python/langgraph/graphs/

7 8 18 27 28 AI Agent Security & Privacy Guide 2026: Protect Your LangChain Apps | LangChain Tutorials
https://langchain-tutorials.github.io/langchain-security-privacy-2026/

13 14 15 21 22 29 Tools | LangChain Reference
https://reference.langchain.com/python/langchain/tools/